

Федеральное агентство связи

Сибирский государственный университет
телекоммуникаций и информатики

Кафедра ТС и ВС

Лабораторная работа №3

По дисциплине Объектно-ориентированное
программирование

Выполнил: Язиков Денис Александрович

Группа: ИКС-531

Вариант: 24

Проверила: Моренкова Ольга Ильинична

Новосибирск 2026

Лабораторная работа 3

Тема: Работа с двоичными файлами

Цель работы: Научиться создавать бинарный файл из текстовых данных, читать записи и обрабатывать их.

Задание

Вывести ведомость закупок чая из таблицы 7, хранящуюся в двоичном файле. Определить, в какой упаковке чай продаётся лучше. По общему заданию нужны класс записи, исходный текстовый файл и две отдельные программы.

Алгоритм решения

1. В tea.txt записать число записей, тип чая, упаковку, производителя, цену и количество. Использовать семь строк таблицы 7.
2. Программой create прочитать текстовый файл и записать число записей и объекты Tea в tea.bin.
3. Программой process прочитать бинарный файл. Для каждой упаковки сложить количества из всех относящихся к ней записей.
4. Вернуться к первой записи методом seekg, снова прочитать бинарный файл и вывести ведомость. Сумма строки равна цене, умноженной на количество.
5. Найти наибольшее количество и вывести все упаковки с таким результатом. Если количества равны нулю, сообщить об отсутствии продаж.

Схема алгоритма

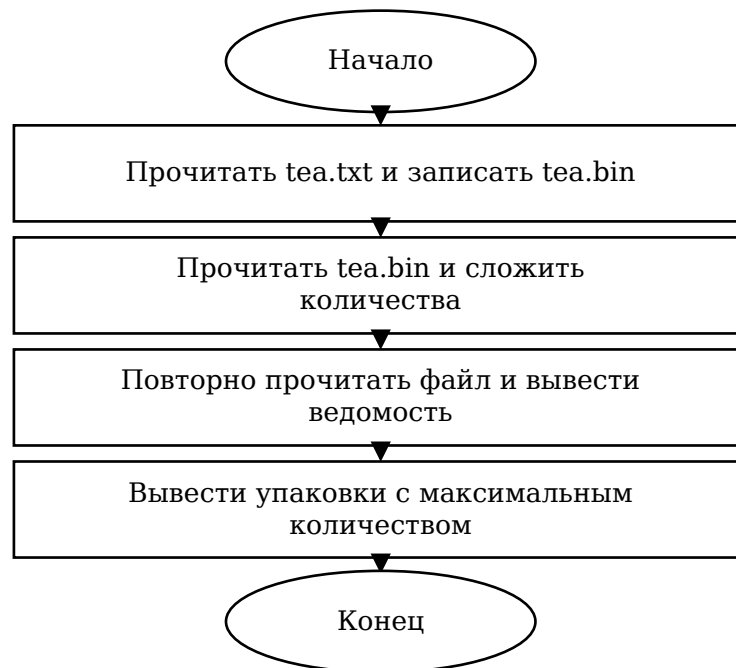


Схема отражает последовательность действий программы.

Пояснения к программе

В качестве критерия «продаётся лучше» выбрано количество проданных единиц, а не выручка. Поэтому разные виды чая в одинаковой упаковке складываются вместе. Цена нужна для вывода суммы закупок.

Поля `type`, `package` и `brand` имеют тип `char[40]`. Здесь нет указателей и объектов `string`, поэтому можно записать байты одного объекта `Tea` через `write`. Метод `read` читает столько же байт. Файл рассчитан на тот же компилятор и платформу; после переноса его можно заново создать из `tea.txt`.

В начале файла записан `int` с числом записей, затем идут объекты `Tea`. Массив `packages` хранит названия упаковок, `counts` - их общие количества. Для подсчёта используется обычный цикл. Проверяются файлы, поля и результат чтения.

Текст программы

lab-03/variant-24/tea.h

```
#ifndef TEA_H
#define TEA_H
#include <iostream>
#include <iomanip>
using namespace std;
class Tea {
public:
    char type[40];
    char package[40];
    char brand[40];
    double price;
    int quantity;
    double total() { return price * quantity; }
    void show() {
        cout << type << " | " << package << " | " << brand << " | ";
        cout << fixed << setprecision(2) << price << " | ";
        cout << quantity << " | " << total() << endl;
    }
};
#endif
```

lab-03/variant-24/create.cpp

```
#include "tea.h"
#include <fstream>
#include <limits>
#include <cmath>
#include <cstdio>
int main() {
    ifstream text("tea.txt");
    if (!text) {
        cout << "Не удалось открыть tea.txt." << endl;
        return 1;
    }
    int n;
    if (!(text >> n) || n < 0 || n > 10000) {
        cout << "Неверное количество записей." << endl;
        return 1;
    }
    text.ignore(numeric_limits<streamsize>::max(), '\n');
    ofstream binary("tea.bin", ios::binary);
    if (!binary) {
        cout << "Не удалось создать tea.bin." << endl;
        return 1;
    }
    binary.write((char*)&n, sizeof(n));
    for (int i = 0; i < n; i++) {
        Tea tea = {};
        text.getline(tea.type, 40);
        text.getline(tea.package, 40);
        text.getline(tea.brand, 40);
        if (!(text >> tea.price >> tea.quantity) ||
            !isfinite(tea.price) || tea.price < 0 || tea.price > 1000000 ||
            tea.quantity < 0 || tea.quantity > 1000000 ||
            tea.type[0] == '\0' || tea.package[0] == '\0' ||
            tea.brand[0] == '\0') {
            cout << "Ошибка в записи " << i + 1 << endl;
            binary.close();
            remove("tea.bin");
            return 1;
        }
        text.ignore(numeric_limits<streamsize>::max(), '\n');
        binary.write((char*)&tea, sizeof(tea));
    }
```

```

    }
    binary.close();
    if (!binary) {
        cout << "Ошибка записи tea.bin." << endl;
        return 1;
    }
    cout << "Создан tea.bin. Записей: " << n << endl;
    return 0;
}

```

lab-03/variant-24/process.cpp

```

#include "tea.h"
#include <fstream>
#include <string>
#include <cmath>
int main() {
    ifstream binary("tea.bin", ios::binary);
    int n;
    if (!binary.read((char*)&n, sizeof(n)) || n < 0 || n > 10000) {
        cout << "Файл tea.bin отсутствует или поврежден." << endl;
        return 1;
    }
    if (n == 0) {
        cout << "Ведомость пуста." << endl;
        return 0;
    }
    string* packages = new string[n];
    long long* counts = new long long[n]();
    int kinds = 0;
    Tea tea = {};
    for (int i = 0; i < n; i++) {
        if (!binary.read((char*)&tea, sizeof(tea)) ||
            tea.type[39] != '\0' || tea.package[39] != '\0' ||
            tea.brand[39] != '\0' || tea.type[0] == '\0' ||
            tea.package[0] == '\0' || tea.brand[0] == '\0' ||
            !isfinite(tea.price) || tea.price < 0 || tea.price > 1000000 ||
            tea.quantity < 0 || tea.quantity > 1000000) {
            cout << "Файл tea.bin поврежден." << endl;
            delete[] packages;
            delete[] counts;
            return 1;
        }
        int j = 0;
        while (j < kinds && packages[j] != tea.package)
            j++;
        if (j == kinds) {
            packages[j] = tea.package;
            kinds++;
        }
        counts[j] += tea.quantity;
    }
    binary.seekg(sizeof(n));
    cout << "Тип | Упаковка | Производитель | Цена, грн | "
        << "Количество | Сумма, грн" << endl;
    double total = 0;
    for (int i = 0; i < n; i++) {
        binary.read((char*)&tea, sizeof(tea));
        tea.show();
        total += tea.total();
    }
    cout << "Всего, грн: " << total << endl;
    long long best = 0;
    cout << "\nПродажи по упаковкам (штук):" << endl;
    for (int i = 0; i < kinds; i++) {
        cout << packages[i] << ": " << counts[i] << endl;
        if (counts[i] > best)
            best = counts[i];
    }
}

```

```

    }
    if (best == 0) {
        cout << "Продаж нет." << endl;
    } else {
        cout << "Лучше продается:" << endl;
        for (int i = 0; i < kinds; i++) {
            if (counts[i] == best)
                cout << packages[i] << " (" << best << " шт.)" << endl;
        }
    }
    delete[] packages;
    delete[] counts;
    return 0;
}

```

Исходный текстовый файл tea.txt

```

7
Зеленый
Пачка
Riston
5.8 50
Черный
Пакетики
Edwin
4.7 23
Красный
Пакетики
Dilmah
6.9 56
Желтый
Пакетики
Edwin
4.4 32
Черный
Пачка
Ahmad
3.9 89
Желтый
Жесть
Dilmah
20.1 23
Черный
Фарфор
Edwin
25.4 76

```

Результаты выполнения

Программы собраны с параметрами `-std=c++11 -Wall -Wextra -pedantic`. Предупреждений компилятора нет. Ниже приведены данные контрольного примера и изображения фактического вывода.

Упаковка	Расчёт количества	Всего
Пачка	50 + 89	139
Пакетики	23 + 56 + 32	111
Жесть	23	23
Фарфор	76	76

```
Создан tea.bin. Записей: 7
```

```
Тип | Упаковка | Производитель | Цена, грн | Количество | Сумма, грн
Зеленый | Пачка | Riston | 5.80 | 50 | 290.00
Черный | Пакетики | Edwin | 4.70 | 23 | 108.10
Красный | Пакетики | Dilmah | 6.90 | 56 | 386.40
Желтый | Пакетики | Edwin | 4.40 | 32 | 140.80
Черный | Пачка | Ahmad | 3.90 | 89 | 347.10
Желтый | Жесть | Dilmah | 20.10 | 23 | 462.30
Черный | Фарфор | Edwin | 25.40 | 76 | 1930.40
Всего, грн: 3665.10
```

```
Продажи по упаковкам (штук):
```

```
Пачка: 139
```

```
Пакетики: 111
```

```
Жесть: 23
```

```
Фарфор: 76
```

```
Лучше продается:
```

```
Пачка (139 шт.)
```

Рисунок 1 - Вывод программы

Лучше продаётся чай в пачках: 139 единиц. Общая сумма семи записей равна 3665.10 грн. Все данные совпадают с ручным подсчётом. Проверены пустой файл с нулём записей, равные максимумы, отсутствие продаж, отсутствие файла, неверная цена и обрыв записи.

Ответы на контрольные вопросы

1. Понятие файла

Файл - именованная последовательность данных на устройстве хранения.

2. Способ описания файла

Для чтения используется `ifstream`, для записи `ofstream`. Режим `ios::binary` включает бинарную обработку.

3. Описание структурного типа

Тип записи объединяет несколько полей. В работе запись задана классом `Tea`.

4. Стандартные функции для работы с файлами

`open` открывает файл, `close` закрывает, `read` читает байты, `write` записывает, `seekg` меняет позицию чтения.

5. Подготовка файла для чтения

Нужно создать `ifstream` с именем файла или вызвать `open` и проверить, что файл открылся.

6. Функции чтения данных

Для текста применяются `>>` и `getline`, для бинарного файла применяется `read`.

7. Как распознать конец файла

По результату попытки чтения. Если `read` не смог получить нужное количество байт, поток переходит в состояние ошибки; `eof` показывает достижение конца.

Вывод

Создан и обработан бинарный файл с семью записями. Ведомость выведена после повторного чтения. Найдена упаковка с максимальным общим количеством.